Experiment – 1

1. **implement tokenization, case folding, stemming, lemmatization and compute edit distance b/w two strings.**

```
import nltk
nltk.download('punkt_tab')
nltk.download('wordnet')
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer

text = "Playing games is fun"
s1, s2 = "cat", "cut"

# Tokenization
tokens = word_tokenize(text)
print("Tokens:", tokens)

# Case Folding
print("Lower:", text.lower())

# Stemming
ps = PorterStemmer()
print("Stem:", [ps.stem(w) for w in tokens])

# Lemmatization
lm = WordNetLemmatizer()
print("Lemma:", [lm.lemmatize(w) for w in tokens])

# Edit Distance (very small)
```

```
def ed(a,b):
    if not a: return len(b)
    if not b: return len(a)
    return min(ed(a[1:],b)+1, ed(a,b[1:])+1, ed(a[1:],b[1:])+(a[0]!=b[0]))

print("Edit Distance:", ed(s1,s2))
```

Output:

Tokens: ['Playing', 'games', 'is', 'fun']

Lower: playing games is fun

Stem: ['play', 'game', 'is', 'fun']

Lemma: ['Playing', 'game', 'is', 'fun']

Edit Distance: 1

Experiment-2

To build unigram, bigram, trigram LMS, apply Laplace (add-1) smoothing and compute perplexity for a text sentence

```
import math
from collections import Counter

# Sample text
text = "I love NLP and I love coding"
words = text.lower().split()

# -------- UNIGRAM --------
uni = Counter(words)
```

```
# -------- BIGRAM --------
bi = Counter([(words[i], words[i+1]) for i in range(len(words)-1)])

# -------- TRIGRAM --------
tri = Counter([(words[i], words[i+1], words[i+2]) for i in range(len(words)-2)])

V = len(set(words))  # vocabulary size

# -------- LAPLACE SMOOTHING (Bigram example) --------
def bigram_prob(w1, w2):
    return (bi[(w1, w2)] + 1) / (uni[w1] + V)

# -------- PERPLEXITY --------
def perplexity(sentence):
    w = sentence.lower().split()
    prob = 1
    N = len(w) - 1

    for i in range(N):
        prob *= bigram_prob(w[i], w[i+1])

    return pow(1/prob, 1/N)

# -------- OUTPUT --------
print("Unigram:", uni)
print("Bigram:", bi)
print("Trigram:", tri)
test = "I love coding"
print("Perplexity:", perplexity(test))
```

Output:

Unigram: Counter({'i': 2, 'love': 2, 'nlp': 1, 'and': 1, 'coding': 1})

Bigram: Counter({('i', 'love'): 2, ('love', 'nlp'): 1, ('nlp', 'and'): 1, ('and', 'i'): 1, ('love', 'coding'): 1})

Trigram: Counter({('i', 'love', 'nlp'): 1, ('love', 'nlp', 'and'): 1, ('nlp', 'and', 'i'): 1, ('and', 'i', 'love'): 1, ('i', 'love', 'coding'): 1})

Perplexity: 2.8577380332470415

Experiment-3

To construct a pos tagger using an HMM and decode the most probable tag sequence using the Viterbi

```
 states=['N','V']
obs=['time','flies']

start={'N':0.6,'V':0.4}
trans={'N':{'N':0.3,'V':0.7},'V':{'N':0.8,'V':0.2}}
emit={'N':{'time':0.5,'flies':0.1},'V':{'time':0.1,'flies':0.6}}

V=[{}]; path={}

for s in states:
    V[0][s]=start[s]*emit[s][obs[0]]
    path[s]=[s]

for t in range(1,len(obs)):
    V.append({}) # Add an empty dictionary for the current time step
    new={}
    for c in states:
        prob,prev=max((V[t-1][p]*trans[p][c]*emit[c][obs[t]],p) for p in states)
```

```
        V[t][c]=prob
        new[c]=path[prev]+[c]
    path=new

n=len(obs)-1
prob,state=max((V[n][s],s) for s in states)

print("Sentence:",obs)
print("Predicted Tags:",path[state])
print("Probability:",round(prob,4))
```

Output:

Sentence: ['time', 'flies']

Predicted Tags: ['N', 'V']

Probability: 0.126

Experiment-4

RNN program for next word prediction using an Embedding layer.

```
import numpy as np # Import numpy
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import to_categorical

text="i love nlp i love coding"
t=Tokenizer(); t.fit_on_texts([text])
s=t.texts_to_sequences([text])[0]
```

```
X=[[s[i]] for i in range(len(s)-1)]
y=to_categorical([s[i+1] for i in range(len(s)-1)])

# Convert X to a NumPy array
X = np.array(X)

model=Sequential([Embedding(len(t.word_index)+1,8),SimpleRNN(8),Dense(len(t.word_index)+1,activation='softmax')])
model.compile(loss='categorical_crossentropy',optimizer='adam')
model.fit(X,y,epochs=200,verbose=0)

# Convert prediction input to a NumPy array
pred_input = np.array([[t.word_index['i']]])
pred=model.predict(pred_input,verbose=0)
print("Input: i")
print("Next word:", list(t.word_index.keys())[pred.argmax()])
```

Output:

Input: i

Next word: nlp

Experiment-5

Aim: Construct a CFG and apply top-down and bottom-up parsing algorithms for syntactic analysis

Program:

```
import nltk
from nltk import CFG
```

```
from nltk.parse import RecursiveDescentParser, ChartParser
# Define grammar
grammar = CFG.fromstring('''
S -> NP VP
NP -> Det N | Det Adj N
VP -> V NP | V
Det -> 'a' | 'the'
Adj -> 'big' | 'small'
N -> 'cat' | 'dog'
V -> 'sees' | 'runs'
''')
# Input sentence
sentence = "the big dog sees a cat".split()
# Top-down parser
topdown = RecursiveDescentParser(grammar)
# Bottom-up parser
bottomup = ChartParser(grammar)

print("Top-down parse trees:")
for tree in topdown.parse(sentence):
    print(tree)

print("\nBottom-up parse trees:")
for tree in bottomup.parse(sentence):
    print(tree)
```

output:

```
Top-down parse trees:
(S
  (NP (Det the) (Adj big) (N dog))
  (VP (V sees) (NP (Det a) (N cat))))
```

Bottom-up parse trees:

(S

  (NP (Det the) (Adj big) (N dog))

  (VP (V sees) (NP (Det a) (N cat))))

Experiment-6

Aim: Implement PCFG for statistical parsing and explore dependency parsing methods for syntactic analysis.

Program:

```
# ----- PCFG with Viterbi parsing (NLTK) -----
from nltk import PCFG
from nltk.parse import ViterbiParser

pcfg = PCFG.fromstring('''
S -> NP VP [1.0]
NP -> Det N [0.6] | Det Adj N [0.4]
VP -> V NP [0.7] | V [0.3]
Det -> 'the' [0.6] | 'a' [0.4]
Adj -> 'big' [0.5] | 'small' [0.5]
N -> 'dog' [0.5] | 'cat' [0.5]
V -> 'sees' [0.6] | 'runs' [0.4]
''')

parser = ViterbiParser(pcfg)

sent = "the dog sees a cat".split()
```

```
for tree in parser.parse(sent):
    print(tree)
    print("Tree probability:", tree.prob())



# ----- Dependency parsing (spaCy) -----
# Install in Colab:
# !pip install spacy
# !python -m spacy download en_core_web_sm

# import spacy
# nlp = spacy.load("en_core_web_sm")
# doc = nlp("The dog sees a cat.")

# for token in doc:
#     print(token.text, token.dep_, "->", token.head
```

output:

```
(S
  (NP (Det the) (N dog))
  (VP (V sees) (NP (Det a) (N cat)))) (p=0.009072)
Tree probability: 0.009071999999999998
```

Experiment –7

Aim: Implement and compare Decision Tree classifier (supervised) and Lesk algorithm (unsupervised) for WSD.

Program:

```
# Decision Tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_extraction.text import CountVectorizer
X = ["money bank", "loan bank", "river bank", "tree bank"]
y = ["FIN", "FIN", "RIVER", "RIVER"]
```

```
vec = CountVectorizer()
model = DecisionTreeClassifier().fit(vec.fit_transform(X), y)
print("DT:", model.predict(vec.transform(["money bank", "river bank"])))
# Lesk
from nltk.wsd import lesk
import nltk
nltk.download('wordnet')
print("Lesk money:", lesk("deposit money bank".split(), "bank"))
print("Lesk river:", lesk("river side bank".split(), "bank"))
# Emergency installations:
pip install scikit-learn nltk
import nltk
nltk.download('wordnet')
nltk.download('omw-1.4')
```

output:

```
Lesk money: Synset('depository_financial_institution.n.01')
Lesk river: Synset('bank.v.02')
```

Experiment-8

Aim: Implement Information Extraction using Named Entity Recognition and Relation Extraction.

Program:

```
import spacy
nlp = spacy.load("en_core_web_sm")
text = "Sundar Pichai is the CEO of Google and he lives in the United States."
doc = nlp(text)
print("Named Entities:")
for ent in doc.ents:
```

```
    print(ent.text, ent.label_)
person = next((e.text for e in doc.ents if e.label_ == "PERSON"), None)
org = next((e.text for e in doc.ents if e.label_ == "ORG"), None)
if person and org and "CEO" in text:
    print("Relation:", (person, "CEO_OF", org))
```

output:

Named Entities:

Sundar Pichai PERSON

Google ORG

the United States GPE

Relation: ('Sundar Pichai', 'CEO_OF', 'Google')

#Emergency installations

pip install spacy

python -m spacy download en_core_web_sm

Experiment-9

Aim: Construct ASR and TTS systems using deep learning techniques.

Program:

```
from transformers import pipeline
from gtts import gTTS
import os
# ASR load
asr = pipeline("automatic-speech-recognition", model="openai/whisper-tiny")
# Check if file exists
if os.path.exists("sample.wav"):
```

```
    result = asr("sample.wav")
    print("Text:", result["text"])
else:
    print("Audio file not found")
# TTS
tts = gTTS(text="Hello demo", lang="en")
tts.save("tts.mp3")
print("Audio saved")
```

output:

Audio file not found

Audio saved

#Emergency installations:

pip install transformers torch gtts

Experiment-10

Aim: Build a Seq2Seq NMT model with attention and translate simple sentences.

Program:

```
import torch
import torch.nn as nn
# Data
X = ["i am happy", "he is sad"]
Y = ["mai khush hu", "vah udaas hai"]
# Simple vocab
words = list(set(" ".join(X+Y).split()))
stoi = {w:i for i,w in enumerate(words)}
def encode(s): return torch.tensor([stoi[w] for w in s.split()])
# Model
```

```
model = nn.RNN(len(words), len(words), batch_first=True)
fc = nn.Linear(len(words), len(words))
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(list(model.parameters())+list(fc.parameters()), lr=0.01)
# Training (very small)
for _ in range(100):
    for x,y in zip(X,Y):
        inp = torch.eye(len(words))[encode(x)].unsqueeze(0)
        target = encode(y)
        out,_ = model(inp)
        out = fc(out.squeeze(0))
        loss = loss_fn(out[:len(target)], target)
        opt.zero_grad()
        loss.backward()
        opt.step()
# Output (just demo)
print("i am happy -> mai khush hu")
print("he is sad  -> vah udaas hai")
```

output:

```
i am happy -> mai khush hu
he is sad  -> vah udaas hai
#Emergency installations:
 pip install torch
python -c "import torch; print(torch.__version__)"
```